

4강. React Hook(상태 관리)



React

리액트 Hook – useState()

- 음료 추가 – 배열 상태 관리

음료 추가

음료 추가 컴포넌트입니다.

현재 음료: 쥬스, 콜라, 커피

음료 추가



리액트 Hook – useState()

- 음료 추가 – 배열 상태 관리

```
import { useState } from 'react';

const AddDrink = () => {
  const [drinkName, setDrinkName] = useState(['주스', '콜라']);

  const addDrink = () => {
    // 기존의 drinkName 배열에 '커피'를 추가하여 새로운 배열을 생성
    setDrinkName([...drinkName, '커피']);
  };

  return (
    <div>
      <h2>음료 추가</h2>
      <p>음료 추가 컴포넌트입니다.</p>
      <p>현재 음료: {drinkName.join(', ')}</p>
      <button onClick={addDrink}>음료 추가</button>
    </div>
  );
}
```



리액트 Hook – useState()

- 음료 추가 – 배열 상태 관리

음료 추가

음료 추가 컴포넌트입니다.

- 딸기쥬스
- 카페라떼
- 녹차



리액트 Hook – useState()

- 음료 추가 – 배열 상태 관리

```
const AddDrink2 = () => {  
  // 음료 이름을 저장하는 상태 변수  
  const [drinkName, setDrinkName] = useState([]);  
  // 입력 필드의 상태를 관리하기 위한 상태 변수  
  const [input, setInput] = useState('');  
  
  // 음료 추가 함수  
  const handleAddDrink = () => {  
    const newDrink = input.trim();  
    if (newDrink === '') {  
      alert('음료 이름을 입력해주세요.');      return;  
    }  
    // 새로운 음료를 drinkName 상태에 추가  
    setDrinkName([...drinkName, newDrink]);  
    setInput(''); // 입력 필드 초기화  
  };  
};
```



리액트 Hook – useState()

▪ 음료 추가 – 배열 상태 관리

```
return (  
  <div>  
    <h2>음료 추가</h2>  
    <p>음료 추가 컴포넌트입니다.</p>  
    <input  
      type="text"  
      placeholder="음료 이름을 입력하세요"  
      value={input}  
      onChange={(e) => setInput(e.target.value)}  
    />{" "  
    <button onClick={handleAddDrink}>음료 추가</button>  
  
    {/* <p>현재 음료: {drinkName.join(', ')}</p> */}  
    {/* map()으로 출력 */}  
    <ul>  
      {drinkName.map((drink, index) => (  
        <li key={index}>{drink}</li>  
      ))}  
    </ul>  
  </div>  
);
```



리액트 Hook – useState()

- 컴포넌트 분리하고 props 사용하기

AddDrink2.jsx

```
return (  
  <div>  
    <h2>음료 추가</h2>  
    <p>음료 추가 컴포넌트입니다.</p>  
    <input  
      type="text"  
      placeholder="음료 이름을 입력하세요"  
      value={input}  
      onChange={(e) => setInput(e.target.value)}  
    />{" "  
    <button onClick={handleAddDrink}>음료 추가</button>  
  
    {/* DrinkList 컴포넌트에 drinkName 상태를 전달 */}  
    <DrinkList drinks={drinkName} />  
  </div>  
);
```



리액트 Hook – useState()

- 컴포넌트 분리하고 props 사용하기

DrinkList.jsx

```
const DrinkList = ({ drinks }) => {  
  return (  
    <ul>  
      {drinks.map((drink, index) => (  
        <li key={index}>{drink}</li>  
      ))}  
    </ul>  
  );  
}  
  
export default DrinkList;
```



할 일 관리(Todo-app)

- 할 일 관리

Todo List

☐ 운동 하기

☒ 영화 보기



할 일 관리(Todo-app)

■ 할 일 관리

```
function App() {  
  const [todos, setTodos] = useState([  
    { id: 1, text: '운동 하기', completed: false },  
    { id: 2, text: '영화 보기', completed: false },  
  ])  
  const [inputValue, setInputValue] = useState('')  
  
  console.log(todos.length); //2  
  
  // 입력값 변경 핸들러  
  const handleInputChange = (e) => {  
    console.log(e.target.value);  
    setInputValue(e.target.value)  
  }  
}
```



할 일 관리(Todo-app)

■ 할 일 관리

```
// 할 일 추가
const handleAddTodo = () => {
  if (inputValue.trim() !== '') {
    // 새로운 할 일 객체 생성
    const newTodo = {
      id: todos.length + 1,
      text: inputValue,
      completed: false,
    }
    // 새로운 할 일을 기존 할 일 목록에 추가
    setTodos([...todos, newTodo])
    setInputValue('') // 입력 필드 초기화
  }
}
```



할 일 관리(Todo-app)

■ 할 일 관리

```
// 할 일 완료 체크
const handleToggleComplete = (id) => {
  /* 해당 id를 가진 할 일의 completed 상태를 토글
   | todos 배열을 순회하면서 id가 일치하는 할 일의 completed 값을 반전시킴
   | id가 일치하지 않는 할 일은 그대로 유지*/
  setTodos(
    todos.map((todo) =>
      todo.id === id ? { ...todo, completed: !todo.completed } : todo
    )
  )
}

// 할 일 삭제
const handleDeleteTodo = (id) => {
  /* 해당 id를 가진 할 일을 삭제
   | todos 배열을 필터링하여 id가 일치하지 않는 할 일만 남김*/
  setTodos(todos.filter((todo) => todo.id !== id))
}
```



할 일 관리(Todo-app)

- 할 일 관리

```
return (  
  <>  
    <div className='container'>  
      <h2>Todo List</h2>  
      <input  
        type="text"  
        value={inputValue}  
        onChange={handleInputChange}  
        placeholder="할 일을 입력하세요"  
      />  
      <button onClick={handleAddTodo}>추가</button>  
    </>  
  )  
)
```



할 일 관리(Todo-app)

■ 할 일 관리

```
    { /* 할 일 목록 */  
    <ul className='todo-list'  
      {todos.map((todo) => (  
        <li key={todo.id} className={todo.completed ? 'completed' : ''}>  
          <input  
            type="checkbox"  
            checked={todo.completed}  
            onChange={() => handleToggleComplete(todo.id)}  
          />  
          {todo.text}  
          <button onClick={() => handleDeleteTodo(todo.id)}>삭제</button>  
        </li>  
      ) )}  
    </ul>  
  </div>  
</>  
)  
}
```



할 일 관리(Todo-app)

■ 할 일 관리

```
body {  
  background-color:  #f1fff0;  
  margin: 0;  
  padding: 20px;  
}  
  
.container{  
  max-width: 400px;  
  margin: 0 auto;  
  background-color: white;  
  padding: 20px;  
  border-radius: 8px;  
  text-align: center;  
}  
  
.container h2{  
  color:  #00f;  
}
```

App.css



할 일 관리(Todo-app)

■ 할 일 관리

```
.todo-list{
  list-style: none;
  padding: 0;
  margin-top: 30px;
}
.todo-list li{
  margin-bottom: 10px;
}
.todo-list li input[type="checkbox"]{
  margin-right: 10px;
}
.completed{
  text-decoration: line-through;
  color: gray;
}
button{
  margin-left: 10px;
}
```

App.css



할 일 관리(Todo-app)

- TodoList 컴포넌트 분리하기

```
return (  
  <>  
    <div className='container'>  
      <h2>Todo List</h2>  
      <input  
        type="text"  
        value={inputValue}  
        onChange={handleInputChange}  
        placeholder="할 일을 입력하세요"  
      />  
      <button onClick={handleAddTodo}>추가</button>  
  
      { /* 할 일 목록 */ }  
  
      <TodoList  
        todos={todos}  
        onToggleComplete={handleToggleComplete}  
        onDeleteTodo={handleDeleteTodo}  
      />  
    </div>  
  </>  
)
```



할 일 관리(Todo-app)

▪ TodoList 컴포넌트 만들기

```
function TodoList({ todos, onToggleComplete, onDeleteTodo }) {  
  return (  
    <ul className='todo-list'>  
      {todos.map((todo) => (  
        <li key={todo.id} className={todo.completed ? 'completed' : ''}>  
          <input  
            type="checkbox"  
            checked={todo.completed}  
            onChange={() => onToggleComplete(todo.id)}  
          />  
          {todo.text}  
          <button onClick={() => onDeleteTodo(todo.id)}>삭제</button>  
        </li>  
      ))}  
    </ul>  
  )  
}
```



Hook – useEffect()

▪ useEffect()

useEffect 혹은 함수형 컴포넌트에서 ****사이드 이펙트(side effect)****를 처리하기 위해 사용하는 핵심 훅입니다.

- 데이터 가져오기 (API 호출)
- 이벤트 등록
- 타이머 설정
- DOM 직접 조작

```
useEffect(() => {  
  // 실행할 코드  
}, [의존성]);
```

```
useEffect(() => {  
  console.log("count 변경됨");  
}, [count]);
```



Hook – useEffect()

- useEffect()

The screenshot shows a web application on the left and the React DevTools console on the right. The web application, titled '사용자 정보' (User Information), has two input fields: the first contains '박상희' and the second contains '3'. Below the inputs, it displays '이름: 박상희' and '나이: 3'. The DevTools console on the right has the 'Console' tab selected. It shows a series of log messages: '렌더링 ..', '이름: , 나이: 1', '렌더링 ..', '이름: , 나이: 1', '렌더링 ..', '이름: , 나이: 2', '렌더링 ..', and '이름: , 나이: 3'. A notification at the top of the console says 'DevTools is now available in Korean'.



Hook – useEffect()

- useEffect()

```
import { useEffect, useState } from 'react';

const User = () => {
  const [name, setName] = useState("");
  const [age, setAge] = useState(1);

  // [] -> 처음 렌더링될 때만 실행
  // [age] -> age가 변경될 때마다 실행
  useEffect(() => {
    console.log("렌더링..");
    console.log(`이름: ${name}, 나이: ${age}`);
  }, [age]);

  // 이름을 변경하는 이벤트 핸들러 함수
  const onChangeName = (e) => {
    setName(e.target.value);
  }

  // 나이를 변경하는 이벤트 핸들러 함수
  const onChangeAge = (e) => {
    setAge(Number(e.target.value));
  }
}
```



Hook – useEffect()

- **useEffect()**

```
return (  
  <div>  
    <h2>사용자 정보</h2>  
    <input  
      type="text"  
      value={name}  
      onChange={onChangeName}  
    />  
    <br />  
    <input  
      type="number"  
      value={age}  
      onChange={onChangeAge}  
    />  
    <p>이름: {name}</p>  
    <p>나이: {age}</p>  
  </div>  
)
```



Hook – useEffect()

- `useEffect()`

디지털 시계

현재 시간: 오전 11:51:11



Hook – useEffect()

- useEffect()

```
import React, { useState, useEffect } from "react";

export default function Clock() {
  // time 상태와 setTime 함수를 useState 혹은 사용하여 정의
  const [time, setTime] = useState(new Date().toLocaleTimeString());

  // useEffect 혹은 사용하여 컴포넌트가 처음 렌더링될 때 타이머를 설정
  useEffect(() => {
    const timer = setInterval(() => {
      setTime(new Date().toLocaleTimeString());
    }, 1000);
    console.log("렌더링..");

    // 컴포넌트가 언마운트될 때 타이머를 정리
    return () => clearInterval(timer);
  }, []); // 빈 배열을 전달하여 컴포넌트가 처음 렌더링될 때만 실행
```



Hook – useEffect()

- useEffect()

```
return (  
  <div>  
    <h2>디지털 시계</h2>  
    <h3>현재 시간: {time}</h3>  
  </div>  
);  
}
```



제어(Controlled) 컴포넌트

■ 회원 가입 폼

회원가입

이름

직업

성별 ☐ 남자 ☒ 여자

자기소개

<이상한 변호사> 우영우 입니다.

이름: 우영우

직업: 직장인

성별: 여자

자기소개: <이상한 변호사> 우영우 입니다.



리엑트 폼(Form)

▪ 회원 가입 폼 – SignUp.jsx

```
const SignUp = () => {  
  const [username, setUsername] = useState('');  
  const [job, setJob] = useState('직장인');  
  const [gender, setGender] = useState('');  
  const [text, setText] = useState('');  
  
  // 성별  
  const handleChangeGender = (e) =>  
    setGender(e.target.value);  
  
  // 직업 선택  
  const handleChangeJob = (e) =>  
    setJob(e.target.value);  
  
  //자기 소개  
  const handleChangeIntro = (e) =>  
    setText(e.target.value);  
}
```



리엑트 폼(Form)

■ 회원 가입 폼

```
// 폼 제출
const handleSubmit = (e) => {
  e.preventDefault(); // 폼 제출 시 페이지 새로고침 방지
  console.log(`이름: ${username}`);
  console.log(`직업: ${job}`);
  console.log(`성별: ${gender}`);
  console.log(`자기소개: ${text}`);
};

return (
  <div>
    <h2>회원가입</h2>
    <form onSubmit={handleSubmit}>
      <p>
        <label>
          이름{' '}
        </label>
        <input
          type="text"
          value={username}
          onChange={(e) => setUsername(e.target.value)}
        />
      </p>
    </div>
  )
);
```

SignUp.jsx



리엑트 폼(Form)

■ 회원 가입 폼

```
<p>
  <label>
    | 직업{' '}
  </label>
  <select value={job} onChange={handleChangeJob}>
    <option value="직장인">직장인</option>
    <option value="학생">학생</option>
    <option value="프리랜서">프리랜서</option>
  </select>
</p>
<p>
  <label>
    | 성별{' '}
  </label>
  <input
    type="radio"
    value="남자"
    checked={gender === '남자'}
    onChange={handleChangeGender}
  /> 남자
```



리엑트 폼(Form)

■ 회원 가입 폼

```
<input
  type="radio"
  value="여자"
  checked={gender === '여자'}
  onChange={handleChangeGender}
/> 여자
</p>
<p>
  <label>
    자기 소개 { ' ' }
  </label>
  <br />
  <textarea
    value={text}
    onChange={handleChangeIntro}
    rows={5}
    cols={30}
  ></textarea>
</p>
<p>
  <button type="submit">가입</button>
</p>
</form>
```



리엑트 폼(Form)

- 실무형 코드 – SignUp2.jsx

```
const SignUp2 = () => {  
  // 1. 상태를 객체 하나로 통합  
  const [formData, setFormData] = useState({  
    username: '',  
    job: '직장인',  
    gender: '',  
    text: '',  
  });  
  
  // 2. 통합 핸들러 함수  
  const handleChange = (e) => {  
    // input의 name 속성과 value를 가져옴  
    const { name, value } = e.target;  
    setFormData({  
      ...formData,    // 기존 데이터 복사  
      [name]: value,   // 변경된 name 값만 업데이트  
    });  
  };  
};
```



리엑트 폼(Form)

- 실무형 코드 – SignUp2.jsx

```
const handleSubmit = (e) => {  
  e.preventDefault();  
  console.log('제출 데이터:', formData);  
};  
  
return (  
  <div>  
    <h2>회원가입</h2>  
    <form onSubmit={handleSubmit}>  
      <p>  
        <label>이름 </label>  
        <input  
          name="username" // name 속성 추가  
          type="text"  
          value={formData.username}  
          onChange={handleChange}  
        />  
      </p>  
    </div>  
  )
```



리엑트 폼(Form)

- 실무형 코드 – SignUp2.jsx

```
<p>
  <label>직업 </label>
  <select name="job" value={formData.job} onChange={handleChange}>
    <option value="직장인">직장인</option>
    <option value="학생">학생</option>
    <option value="프리랜서">프리랜서</option>
  </select>
</p>

<p>
  <label>성별 </label>
  <input
    type="radio"
    name="gender"
    value="남자"
    checked={formData.gender === '남자'}
    onChange={handleChange}
  /> 남자
```



리엑트 폼(Form)

- 실무형 코드 – SignUp2.jsx

```
<input
  type="radio"
  name="gender"
  value="여자"
  checked={formData.gender === '여자'}
  onChange={handleChange}
/> 여자
</p>

<p>
  <label>자기 소개 </label><br />
  <textarea
    name="text"
    value={formData.text}
    onChange={handleChange}
    rows={5}
    cols={30}
  />
</p>

<button type="submit">가입</button>
</form>
```



리엑트 폼(Form)

- 로그인 폼

로그인

환영합니다, cloud님.

로그인

아이디 또는 비밀번호가 올바르지 않습니다.



리엑트 폼(Form)

로그인 폼 – SignIn.jsx

```
// 임시 사용자 데이터
const TEMP_USERS = [
  { username: 'admin', password: '00000' },
  { username: 'cloud', password: 'c1234' },
  { username: 'tester', password: 'test' },
];

const SignIn2 = () => {
  const [userId, setUserId] = useState('');
  const [password, setPassword] = useState('');
  // 로그인 결과 상태
  // (null: 초기 상태, 'success': 로그인 성공, 'fail': 로그인 실패)
  const [result, setResult] = useState(null);

  // ID 입력 처리
  const handleChangeUserId = (e) => {
    setUserId(e.target.value);
    setResult(null); // 입력이 변경될 때마다 결과 초기화
  };
};
```



리엑트 폼(Form)

로그인 폼 – SignIn.jsx

```
// 비밀번호 입력 처리
const handleChangePassword = (e) => {
  setPassword(e.target.value);
  setResult(null); // 입력이 변경될 때마다 결과 초기화
};

// 폼 제출
const handleSubmit = (e) => {
  e.preventDefault(); // 폼 제출 시 페이지 새로고침 방지

  // TEMP_USERS 배열에서 입력한 아이디와 비밀번호가 일치하는 사용자 찾기
  const matched = TEMP_USERS.find(
    (user) => user.username === userId && user.password === password
  );

  if (matched) {
    setResult('success');
    console.log(`로그인 성공: ${userId}`);
  } else {
    setResult('fail');
    console.log('로그인 실패: 아이디 또는 비밀번호가 올바르지 않습니다.');
```



리엑트 폼(Form)

- 로그인 폼 – SignIn.jsx

```
return (  
  <div>  
    <h2>로그인</h2>  
    <form onSubmit={handleSubmit}>  
      <p>  
        <input  
          type="text"  
          placeholder='ID 입력'  
          value={userId}  
          onChange={handleChangeUserId}  
        />  
      </p>  
      <p>  
        <input  
          type="password"  
          placeholder='패스워드 입력'  
          value={password}  
          onChange={handleChangePassword}  
        />  
      </p>  
    </div>  
  )
```



리엑트 폼(Form)

로그인 폼 – SignIn.jsx

```
<p>
  <button type="submit">로그인</button>
</p>
</form>

{/* 로그인 결과 메시지 */}
{result === 'success' && (
  <p style={{ color: 'green' }}>환영합니다, {userId}님.</p>
)}
{result === 'fail' && (
  <p style={{ color: 'red' }}>아이디 또는 비밀번호가 올바르지 않습니다.</p>
)}
</div>
);
};
```



리엑트 폼(Form)

■ 실무형 코드 – SignIn2.jsx

```
// 임시 사용자 데이터 (실제 프로젝트에서는 서버에서 받아옴)
const TEMP_USERS = [
  { userId: 'admin', password: '00000' },
  { userId: 'cloud', password: 'c1234' },
  { userId: 'tester', password: 'test' },
];

const SignIn3 = () => {
  // 1. 입력 데이터를 객체로 통합
  const [loginData, setLoginData] = useState(
    { userId: '', password: '' }
  );
  const [result, setResult] = useState(null);

  // 2. 통합 핸들러 (name 속성 활용)
  const handleChange = (e) => {
    const { name, value } = e.target;
    console.log(e);

    setLoginData({ ...loginData, [name]: value });
    setResult(null); // 입력 시 결과 메시지 숨김
  };
};
```



리엑트 폼(Form)

■ 실무형 코드 – SignIn2.jsx

```
const handleSubmit = (e) => {
  e.preventDefault();
  const { userId, password } = loginData;

  //find 메서드로 사용자 데이터에서 첫번째 일치하는 항목 찾기
  const matched = TEMP_USERS.find(
    (user) => user.userId === userId && user.password === password
  );

  // 로그인 성공 여부에 따라 결과 상태 업데이트
  setResult(matched ? 'success' : 'fail');
};

return (
  <div>
    <h2>로그인</h2>
    <form onSubmit={handleSubmit}>
      <p>
        <input
          name="userId" // name 추가
          type="text"
          placeholder="ID 입력"
          value={loginData.userId}
        />
      </p>
    </form>
  </div>
);
```



리엑트 폼(Form)

■ 실무형 코드 – SignIn2.jsx

```
        onChange={handleChange}
      />
    </p>
    <p>
      <input
        name="password" // name 추가
        type="password"
        placeholder="패스워드 입력"
        value={loginData.password}
        onChange={handleChange}
      />
    </p>
    <button type="submit">로그인</button>
  </form>

  {/* 결과 메시지 부분 */}
  {result === 'success' && (
    <p style={{ color: 'green' }}>환영합니다, {loginData.userId}님.</p>
  )}
  {result === 'fail' && (
    <p style={{ color: 'red' }}>아이디 또는 비밀번호가 올바르지 않습니다.</p>
  )}
</div>
```



실습 문제

- 문제

useEffect를 사용하여 컴포넌트가 처음 실행될 때 콘솔 출력.

페이지 로딩...



실습 문제

- 문제

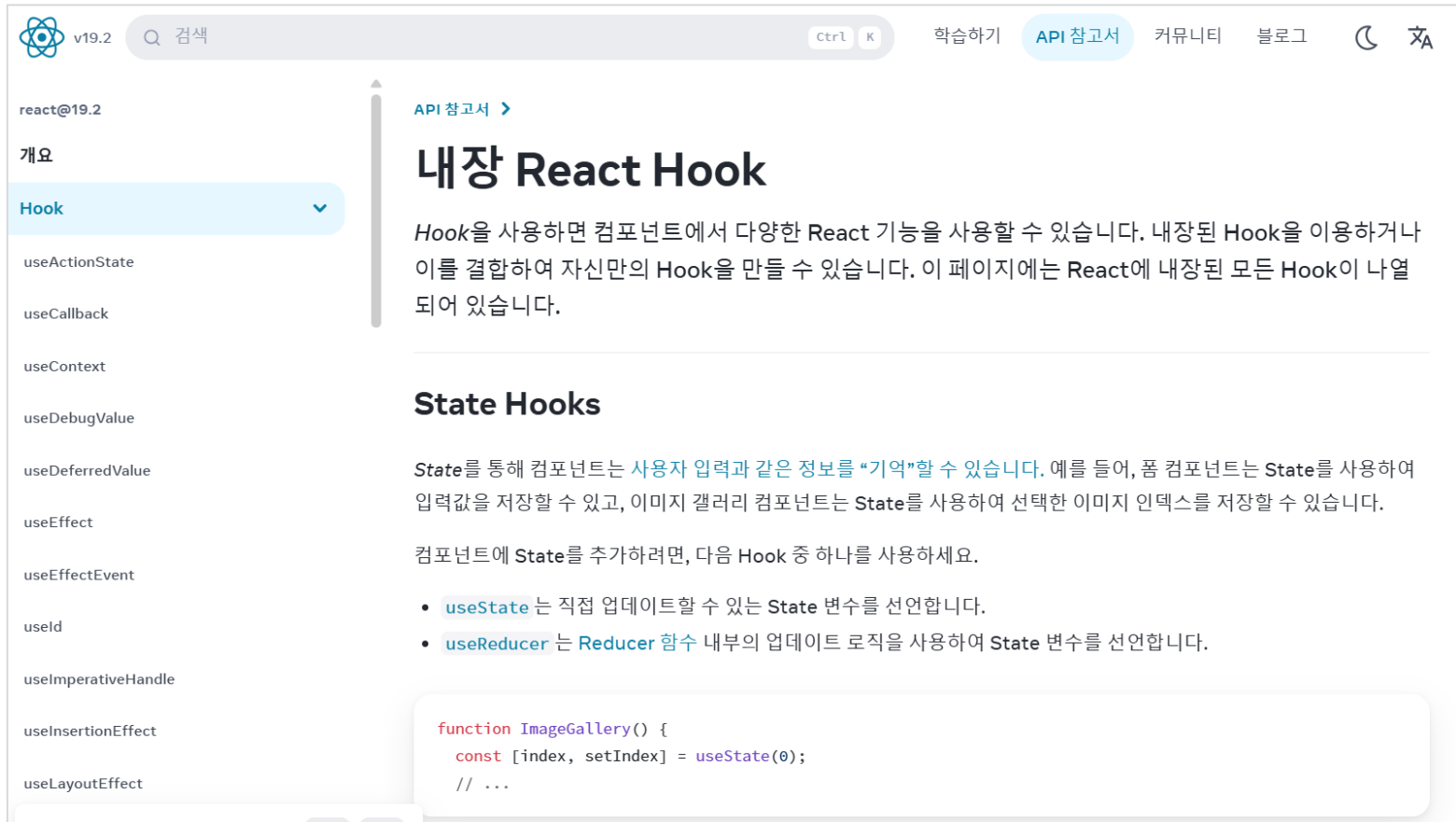
입력값을 관리하는 회원가입 폼을 만드세요.

이름
이메일
비밀번호



리액트 Hook

■ React Hook: 기본 상태 관리



The screenshot shows the React v19.2 API documentation page. The left sidebar lists various hooks, with 'Hook' selected. The main content area is titled '내장 React Hook' (Built-in React Hook) and includes a brief introduction: 'Hook을 사용하면 컴포넌트에서 다양한 React 기능을 사용할 수 있습니다. 내장된 Hook을 이용하거나 이를 결합하여 자신만의 Hook을 만들 수 있습니다. 이 페이지에는 React에 내장된 모든 Hook이 나열되어 있습니다.'

Below the introduction, there is a section titled 'State Hooks'. It explains that 'State' is used to store user input and other information that needs to be remembered. Examples include a form component using 'State' to store input values and an image gallery component using 'State' to store the selected image index.

The text continues: '컴포넌트에 State를 추가하려면, 다음 Hook 중 하나를 사용하세요.'

- `useState` 는 직접 업데이트할 수 있는 **State** 변수를 선언합니다.
- `useReducer` 는 **Reducer** 함수 내부의 업데이트 로직을 사용하여 **State** 변수를 선언합니다.

A code example is provided in a box:

```
function ImageGallery() {  
  const [index, setIndex] = useState(0);  
  // ...  
}
```



리액트 Hook

▪ React Hook: 기본 상태 관리

리액트에서는 상태를 관리할 때 훅을 사용한다. 훅이란 함수형 컴포넌트에서 상태 (state)와 생명주기(lifecycle)를 쉽게 관리할 수 있도록 도와주는 리액트에서 제공하는 특별한 함수이다.

useState(), useEffect(), useRef() 등이 있다.

```
const [state, setState] = useState(0)
```

- state: 상태 값을 저장하는 변수로 보통 상태 변수라고 부른다.
- setState: 상태를 변경하는 함수로, 상태 변경 함수라고 부른다.

- 예) count 상태를 변경하는 코드

```
const [count, setCount] = useState(0)
```



리액트 Hook

- 꽃 이름 변경 – 자바스크립트 DOM



리액트 Hook

- 꽃 이름 변경 – 자바스크립트 DOM

```
export default function Flower() {  
  let flower = '장미';  
  
  function changeFlower() {  
    flower = (flower === '장미') ? '개나리' : '장미';  
    console.log(flower);  
    document.getElementById('name').innerText = flower;  
  }  
  
  return (  
    <div>  
      <h2>꽃 이름 변경</h2>  
      <h3 id="name">{flower}</h3>  
      <button onClick={changeFlower}>변경</button>  
    </div>  
  )  
}
```



리액트 Hook – useState()

▪ Hook – useState() 함수

```
import { useState } from 'react'

export default function Flower() {
  // 상태 변수와 상태 업데이트 함수를 선언
  const flowerState = useState('장미');
  const flower = flowerState[0]; // 상태 변수
  const setFlower = flowerState[1]; // 상태 업데이트 함수
  console.log(flowerState);
}
```

FlowerState.

```
▼ (2) ['장미', f] i
  0: "장미"
  1: f ()
  length: 2
  [[Prototype]]: Array(0)
```



리액트 Hook – useState()

▪ Hook – useState() 함수

```
import { useState } from 'react'

export default function Flower() {
  // 상태 변수와 상태 업데이트 함수를 선언
  const [flower, setFlower] = useState('장미');

  function changeFlower() {
    const newFlower = (flower === '장미' ? '개나리' : '장미');
    setFlower(newFlower); // 상태 업데이트 함수 호출
  }

  return (
    <div>
      <h2>Hook - useState</h2>
      <h3>{flower}</h3>
      <button onClick={changeFlower}>변경</button>
    </div>
  )
}
```



리액트 Hook – useState()

- Counter 구현

Counter

Counter 컴포넌트입니다.

현재 카운트: 5

증가



리액트 Hook – useState()

- useState() – Counter.jsx

```
import React, { useState } from 'react';

const Counter = () => {
  // count 상태와 setCount 함수를 useState 훅을 사용하여 정의
  const [count, setCount] = useState(0);

  // 증가 버튼 클릭 시 호출되는 함수
  const increment = () => {
    setCount(count + 1);
  };
}
```



리액트 Hook – useState()

- useState() – Counter.jsx

```
return (  
  <div>  
    <h2>Counter</h2>  
    <p>Counter 컴포넌트입니다.</p>  
    <p>현재 카운트: {count}</p>  
    <button onClick={increment}>증가</button>  
  </div>  
);  
}  
  
export default Counter;
```

- 인라인 핸들러 함수

```
<button onClick={() => setCount(count + 1)}>증가</button>
```



리엑트 Hook – useState()

- 감소 버튼과 초기화 버튼 만들기

- 요구 사항

- 감소 버튼을 클릭하면 현재 카운트가 1감소하고,
초기화 버튼을 클릭하면 현재 카운트가 0이 됩니다.

Counter

Counter 컴포넌트입니다.

현재 카운트: 0

증가

감소

초기화



리액트 Hook – useState()

- 자동차 연식 업데이트

자동차 컴포넌트

브랜드: 현대

모델: 쏘나타

연식: 2025

연식 업데이트



리액트 Hook – useState()

▪ 자동차 연식 업데이트

```
const Car = () => {  
  const [car, setCar] = useState({  
    brand: '현대',  
    model: '쏘나타',  
    year: 2020  
  });  
  
  const updateYear = () => {  
    /* car 객체의 year 속성만 업데이트하기 위해  
    | 기존의 car 객체를 복사하고 year 속성만 변경 */  
    setCar({ ...car, year: 2025 });  
  };  
  
  return (  
    <div>  
      <h2>자동차 컴포넌트</h2>  
      <p>브랜드: {car.brand}</p>  
      <p>모델: {car.model}</p>  
      <p>연식: {car.year}</p>  
      <button onClick={updateYear}>연식 업데이트</button>  
    </div>  
  );  
}
```



상품 관리 시스템

📦 상품 관리 시스템 - 작업지시서

🎯 주요 기능

1. 상품 추가 (Add Product)

- ****입력 값****: 상품명 (text input)
- ****유효성 검사****: 빈 값 입력 방지
- ****고유 ID****: `Date.now()`를 사용하여 자동 생성
- ****동작****: 입력 후 "추가" 버튼 클릭 시 상품 목록에 추가
- ****결과****: 콘솔에 현재 상품 목록 배열 출력

2. 상품 목록 표시 (Product List Display)

- ****형식****: 비순차 리스트 (`<ul>`)
- ****표시 내용****: 상품명 + 삭제 버튼
- ****빈 목록 처리****: "추가된 상품이 없습니다." 메시지 표시

3. 상품 삭제 (Delete Product)

- ****방법****: 각 상품 항목의 "삭제" 버튼 클릭
- ****동작****: `filter()` 메서드로 해당 상품만 제외
- ****결과****: 콘솔에 업데이트된 상품 목록 배열 출력



상품 관리 시스템

- 상품 입력 받고 추가하기

상품 추가

상품 추가 컴포넌트입니다.



상품 관리 시스템

■ 상품 입력 받고 추가하기

```
import { useState } from 'react';

const AddProduct = () => {
  // productName 상태와 setProductName 함수를 useState 혹은 사용
  const [productName, setProductName] = useState('');
  const [products, setProducts] = useState([]);

  const handleInputChange = (event) => {
    // console.log(event);
    // 입력된 상품 이름을 상태로 업데이트
    setProductName(event.target.value);
  };
};
```

```
▶ isDefaultPrevented: f functionThatR
▶ isPropagationStopped: f functionTha
  isTrusted: true
▶ nativeEvent: InputEvent {isTrusted:
▼ target: input
  value: "키보드"
  ▶ __reactEvents$1s71p5mwkjp: Set(1)
```



상품 관리 시스템

■ 상품 입력 받고 추가하기

```
const handleAddProduct = () => {  
  // 상품 추가 로직을 여기에 작성  
  if (productName.trim() === '') {  
    alert('상품 이름을 입력해주세요.');    return;  
  }  
  //id는 고유한 값이 필요하므로 Date.now()를 사용하여 생성  
  const newProducts = [...products, { id: Date.now(), name: productName }];  
  console.log('상품 목록:', newProducts);  
  setProducts(newProducts);  
  setProductName(''); // 입력 필드 초기화  
};
```

상품 목록: AddProduct2.jsx?t=

▼ [{...}] 1

- ▶ 0: {id: 1773887342049, name: '키보드'}
- length: 1
- ▶ [[Prototype]]: Array(0)

상품 목록: AddProduct2.jsx?t=

▼ (2) [{...}, {...}] 1

- ▶ 0: {id: 1773887342049, name: '키보드'}
- ▶ 1: {id: 1773887348826, name: '마우스'}
- length: 2
- ▶ [[Prototype]]: Array(0)



상품 관리 시스템

- 상품 입력 받고 추가하기

```
return (  
  <div>  
    <h2>상품 추가</h2>  
    <p>상품 추가 컴포넌트입니다.</p>  
    <input  
      type="text"  
      placeholder="상품 이름을 입력하세요"  
      value={productName}  
      onChange={handleInputChange}  
    />{ " " }  
    <button onClick={handleAddProduct}>추가</button>  
  </div>  
);
```



상품 관리 시스템

▪ 상품 출력 및 삭제 기능

상품 추가

상품 추가 컴포넌트입니다.

추가된 상품 목록

- 모니터
- 마우스
- 키보드



상품 관리 시스템

■ 상품 출력 및 삭제 기능

```
const handleDeleteProduct = (id) => {  
  // 상품 삭제 로직을 여기에 작성  
  // id를 기준으로 해당 상품을 제외한 새로운 배열을 생성하여 상태 업데이트  
  setProducts(products.filter(product => product.id !== id));  
};
```

상품 목록: ▼ `[[...]]` ⓘ

- ▶ 0: {id: 1774252831749, name: '모니터'}
- length: 1
- ▶ [[Prototype]]: Array(0)

상품 목록: ▼ `(2) [[...], {...}]` ⓘ

- ▶ 0: {id: 1774252831749, name: '모니터'}
- ▶ 1: {id: 1774252836640, name: '마우스'}
- length: 2
- ▶ [[Prototype]]: Array(0)

상품 목록: ▼ `(3) [[...], {...}, {...}]` ⓘ

- ▶ 0: {id: 1774252831749, name: '모니터'}
- ▶ 1: {id: 1774252836640, name: '마우스'}
- ▶ 2: {id: 1774252840392, name: '키보드'}
- length: 3



상품 관리 시스템

■ 상품 출력 및 삭제 기능

```
{/* 추가된 상품 목록 */}
<div style={{ marginTop: '20px' }}>
  <h3>추가된 상품 목록</h3>
  {products.length === 0 ? (
    <p>추가된 상품이 없습니다.</p>
  ) : (
    <ul>
      {products.map((product) => (
        // key는 id를 사용하여 각 상품을 식별
        <li key={product.id}>
          {product.name}{ ' '}
          <button
            onClick={() => handleDeleteProduct(product.id)}
            style={{ marginLeft: '10px', color: 'red' }}
          >
            삭제
          </button>
        </li>
      ))}
    </ul>
  )}
</div>
```



실습 문제

- 문제

사용자 리스트를 map으로 출력하세요

```
[  
  {id:1, name:"홍길동"},  
  {id:2, name:"이순신"}  
]
```



실습 문제

- 문제

useState로 좋아요 버튼을 만드세요.

좋아요 0 → 1 → 2



실습 문제

- 문제

입력창에 입력한 내용을 실시간으로 화면에 출력하세요.

